

DC2AP: A Dublin Core Application Profile to Analysis Patterns

Lucas Francisco da Matta Vegi, Jugurta Lisboa-Filho, Glauber Luis da Silva Costa,
Alcione de Paiva Oliveira and José Luís Braga

Departamento de Informática
Universidade Federal de Viçosa
Viçosa-MG, Brazil, 36570-000

lucasvegi@gmail.com, jugurta@ufv.br, glaufercosta@gmail.com, alcione@dpi.ufv.br, zeluibraga@ufv.br

Abstract—Analysis patterns are reusable computational artifacts, aimed at the analysis stage of the process of software development. Although the analysis patterns can facilitate the work of analysts and programmers adding value through reuse of proven useful and tested ideas, the access to them is still very poor because of the way they are usually described and made available. In order to reduce these deficiencies, supporting cataloging and encouraging the reuse of analysis patterns, it was proposed the Analysis Patterns Reuse Infrastructure (APRI). This infrastructure comprises a repository of analysis patterns documented through a specific metadata profile and accessed via web services. Based on the proposal of APRI, this article presents the specific metadata profile to the documentation of analysis patterns called Dublin Core Application Profile to Analysis Patterns (DC2AP).

Keywords- *Analysis Patterns; Reuse; Metadata Standards; Dublin Core.*

I. INTRODUCTION

Correction of errors made during the encoding of a software is usually more costly than the correction performed during the stages of analysis and design. The costs for correction of errors increase on each stage and in advanced stages they can be up to 100 times higher than in the early stages [1].

Due to the needs of companies, programmers and analysts are constantly pressured to deliver encoded projects of software as soon as possible, and the analysis stage is often left in the background [2]. This common situation in companies ends up generating software errors identified too late, thus burdening the costs of the final product.

According to Fernandez and Yuan [2], analysis patterns can make the analysis stage faster and more accurate for developers, thus preventing that this important stage of development is ignored. The analysis patterns are reusable computational artifacts, aimed at the analysis stage of the process of software development. According to Fowler [3], the analysis patterns are ideas proved useful in a given practical context and that may be useful in other contexts.

Although the analysis patterns can facilitate the work of analysts and programmers adding value through reuse of proven useful and tested ideas, the access to them is still very poor [4]. So far there is no template to specify the analysis patterns that is widely accepted making each set of analysis

patterns is specified according to the preferences of its authors. In addition to not having a pattern specification, the analysis patterns are normally provided in scientific books and papers which are restricted access means and do not allow the efficient retrieval of patterns performed, for example, through a search tool [4].

In order to minimize these problems of specification and retrieval of analysis patterns, it was proposed the Analysis Patterns Reuse Infrastructure (APRI) [5]. This infrastructure, which was inspired by the Spatial Data Infrastructure (SDI) [6], consists of a repository of analysis patterns, documented in a specific metadata profile and accessed via web services.

This article proposes a specific metadata profile to the documentation of analysis patterns compatible with the APRI [5]. The metadata profile is based on the Dublin Core metadata standard [7] and on the template proposed in [8] and [9] to specify analysis patterns.

The remainder of this paper is organized as follows: Section 2 describes related works to documentation of analysis patterns and to the Dublin Core metadata standard. Section 3 describes the proposed metadata profile. Section 4 presents an example of analysis pattern specified with the proposed metadata profile and Section 5 presents some conclusions and possible future works.

II. RELATED WORK

A. Documentation and Organization of Analysis Patterns

Documentation of analysis patterns is an important way for contextualizing the reuse scope of a pattern and for enabling the sharing of knowledge among designers. However, this documentation is performed in a heterogeneous manner among the authors, since there is no standardized way to specify analysis patterns [4]. There are many approaches to specify analysis patterns, ranging from non-formalized textual descriptions to formalized descriptions based on templates.

Some analysis patterns specified in a non-formalized textual manner can be found in [3] and [10]. This little formal way of describing a analysis pattern affects reuse, because it makes harder for designers to quickly understand the contextual scope of patterns, and, mostly, it limits the retrieval of analysis patterns through computerized search engines. Thus important detailed information for designers may not be

described or even retrieved, thus limiting the spread of these patterns and thereby their potential for reuse.

Analysis patterns have also been described through the use of templates, which are structures with predetermined topics similar to those used to describe design patterns [11]. Usually a template is composed of essential topics such as context, problem, motivation and solution [3], combined with other specific topics defined by their authors.

Some analysis patterns documented through templates can be found in [12] and [13]. Meszaros and Doble [14] present in their work a template composed of topics: name, problem, context, motivation, solution, participants and related patterns. This template was used in [12]. Pantoquilho, Raminhos and Araujo [8] and Raminhos et al. [9] present in their work a proposal of detailed template specifically for documentation of analysis patterns. This template combines common topics used previously by several authors, with new topics aimed at describing the analysis patterns more broadly.

Besides adequate documentation, another important factor to increase the potential for reuse of analysis patterns is the way they are organized and therefore available because before a pattern is applied to a project, the designer needs to know of its existence and then select it [4]. Usually analysis patterns are organized together forming collections of patterns, and in one collection, usually the patterns are documented homogeneously by the same author, although this is not a rule.

The collection of analysis patterns can have different formats, such as books, articles and websites, and they can still be classified as pattern languages and pattern catalogues [4]. The pattern languages are basically collections of analysis patterns aimed at solving a specific problem. In a pattern language, the patterns are related to each other and must follow application rules, for example, the order in which they must be applied to solve the problem in question [4]. The pattern catalogues are collections of analysis patterns not necessarily related, but organized based on criteria in common and searchable. Fowler's book [3] is an example of an analysis pattern catalog, because the organization of the patterns described in that book was obtained from groups of patterns with application domains in common and they may be found by potential users through a table of contents [4].

B. Dublin Core Metadata Initiative

Metadata are data about data, i.e., information that makes it possible to know the resources of the data. They can be used to standardize data representation based on the description of their authors, quality levels, application domains and other elements, thereby encouraging the appropriate reuse of data [6].

The metadata standards are metadata structures used to describe the data. According to the domain of the data to be described, these description structures may have variations compared to other domains due to peculiarities of each of them. Thus, several metadata standards have already been proposed in order to meet the specific needs of some domains. Examples of metadata standards for specific domains can be found in [15] and [16].

The Dublin Core metadata standard [7] appeared in 1995 from a workshop held in Dublin city in the U.S. state of Ohio. This event brought together professionals from several fields of knowledge in order to establish a generic metadata standard composed of a small set of recurring elements in all areas [17]. This metadata standard has two levels, Simple Dublin Core and Qualified Dublin Core.

The Simple Dublin Core consists of fifteen elements, as the Qualified Dublin Core has seven additional elements that allow more detailed descriptions of data. Beyond the twenty-two elements that compose the levels of the Dublin Core standard, it has many element refinements that can be used to specialize the semantics of an element in certain situations and thus facilitate the discovery of the data [18].

A major advantage of the Dublin Core standard is its versatility. Although it is very simple and does not provide enough resources to describe data of complex domains [17], it can be specialized from the creation of application profiles for specific domains [6].

Coyle and Baker [19] describe in their work the basic steps for creating a Dublin Core application profile. An example of a Dublin Core application profile for specific domain can be found in [20].

III. A DUBLIN CORE APPLICATION PROFILE TO ANALYSIS PATTERNS

The Dublin Core Application Profile to Analysis Patterns (DC2AP) was developed based on the template proposed in [8] to specify analysis patterns. The main objectives of DC2AP are to improve retrieval and reuse of analysis patterns by means of a description that allows a more precise treatment performed by a computer, providing detailed information about the analysis patterns that were not retrieved by search engines.

A. Mapping between Dublin Core Metadata Elements and Pantoquilho's Analysis Pattern Template

In contrast to the Dublin Core metadata standard, which is generic and therefore aimed to document resources of several domains, the template proposed in [8] by Pantoquilho et al. is designed specifically for the documentation of analysis patterns, so it is rich in specific details of this domain. Due to such level of detail, this template was chosen to be used as a basis for the creation of DC2AP.

An important task for the creation of DC2AP was the realization of a mapping between the elements proposed by the Dublin Core and elements of the template proposed in [8]. In this mapping process, elements of both structures were compared and classified based on their semantic correspondences and some conceptual conflicts were identified. These conflicts are characterized by similar concepts that are expressed differently by each of the mapped structures. After identifying and resolving the existing conceptual conflicts between structures it became possible to combine the elements of Dublin Core and Pantoquilho's template, thus creating a single structure free of redundancies and semantic inconsistencies. Table 1 shows the result of the mapping between the Dublin Core standard and Pantoquilho's template. Although Table 1 shows only the mapping between the

elements of Simple Dublin Core and the Pantoquilho's template, the elements contained in Qualified Dublin Core were also considered in this comparative process, but none of them had equivalents in the template used.

Several mappings between elements of the Dublin Core and elements of other structures have already been performed and made available in the literature. An example of such mapping is presented in [21].

B. Addition of New Metadata Elements and Creation of Application Rules

From the mapping described above, all equivalent elements have been identified and combined, thereby allowing that the Pantoquilho's template to be merged with Dublin Core, giving rise to the basic structure of the DC2AP.

Most elements of the Pantoquilho's template that had direct equivalent mapping became element refinements of others from the Dublin Core. This happened because Dublin Core elements are generic and therefore require specializations to compose an application profile for a specific domain. These necessary specializations were made by the elements of the chosen template [8].

During the merge process of the structures in question, two elements from Pantoquilho's template were discarded. The element "Applicability" was discarded because it has semantics very similar to the element "Problem" and therefore was considered redundant. The element "Structural adjustments" was discarded for not fitting in the context of an application profile of metadata, where application rules are well defined, not being necessary to document structural adjustments performed during use of the profile.

Following the fusion of structures, some elements have undergone semantic adjustments and new ones were proposed to complete the set of elements that composes DC2AP. Table 2 presents all the elements that make up the profile proposed by this work.

TABLE I. MAPPING DUBLIN CORE TO PANTOQUILHO'S ANALYSIS PATTERN TEMPLATE

Simple Dublin Core element	Pantoquilho's Template element
Title	1. Name 2. Also Known As
Creator	3. History *
Subject	7. Context
Description	5. Problem 6. Motivation 7. Context 8. Applicability 14. Examples * 18. Known Uses *
Publisher	No equivalent
Contributor	3. History *
Date	3. History *
Type	No equivalent
Format	No equivalent
Identifier	1. Name *
Source	15. Related Patterns *
Language	No equivalent

Simple Dublin Core element	Pantoquilho's Template element
Relation	13. Anti-Patterns Trap * 15. Related Patterns * 16. Design Patterns *
Coverage	No equivalent
Rights	No equivalent
No equivalent	4. Structural adjustments 9. Requirements 9.1. Functional requirements 9.2. Non-functional requirements 9.3. Dependencies and contributions 9.4. Conflict identification & guidance to resolution 9.5. Priorities 9.6. Participants 10. Modelling 10.1. Structure 10.1.1. Class diagram 10.1.2. Class description 10.2. Behaviour 10.2.1. Collaboration or sequence diagrams 10.2.2. Activity diagrams 10.2.3. State diagrams 10.3. Solution Variants 11. Resulting context 12. Consequences 17. Design guidelines

* Partly equivalent.

As shown in Table 2, DC2AP has some elements for version control of documented patterns and others for the sharing of experiences of use. These features were incorporated into this profile to allow the creation of dynamic collections of analysis patterns, where new improved versions of the patterns can be proposed from the collaboration of experience of usage of them. Moreover, all the versions of the analysis patterns may be related to each other, thereby providing the creation of a repository of analysis patterns rich in details. These resources allow potential users to retrieve the version that best meets their needs more efficiently. All these characteristics are consistent with the proposal of the APRI [5].

The analysis patterns usually have rules controlling their application. After defining the elements that compose DC2AP, it was proposed rules on the obligation, occurrence and type of value of each of the proposed elements. These rules are presented in Table 2 by acronyms, described at the table end.

Due to limited space, the semantic description of each of the elements that compose DC2AP, as well as some details of the rules for applying them are not presented in this paper. However a detailed technical description of this application profile can be obtained at [22].

TABLE II. DC2AP ELEMENTS AND RULES OF APPLICATION

DC2AP Element and their Rules of application	New
1. Identifier [M] [S] [UNS]	
2. Title [M] [S] [St] 2.1. Alternative Title [O] [Mu] [St]	
3. Creator [M] [Mu] [St]	

DC2AP Element and their Rules of application			New
4. Subject [M] [Mu] [St]			
5. Description [M] [S] [N]	5.1. Problem [M] [S] [St]		
	5.2. Motivation [M] [Mu] [St]	5.2.1. Example [M] [Mu] [St]	
		5.2.2. Known Uses** [O] [Mu] [St]	
	5.3. Context [M] [S] [St]		
6. Publisher [O] [Mu] [St]			
7. Contributor [Cd] [Mu] [St]			
8. Date [M] [S] [N]	8.1. Created [M] [S] [D]		
	8.2. Modified [Cd] [S] [D]		
9. Type [M] [S] [US]	9.1. Notation [M] [S] [St]		YES
10. Format [M] [Mu] [US]			
11. Source [Cd] [S] [UNS]			
12. Language [M] [S] [US]			
13. Relation [Cd] [S] [N]	13.1. Is Version of [Cd] [S] [UNS]		
	13.2. Is Replaced by* [Cd] [Mu] [UNS]		
	13.3. Replaces* [Cd] [Mu] [UNS]		
	13.4. Is Part of [O] [Mu] [UNS]		
	13.5. Has Part [O] [Mu] [UNS]		
	13.6. Is Designed with** [O] [Mu] [UNS]		YES
	13.7. Should Avoid** [O] [Mu] [UNS]		YES
	13.8. Complemented by** [O] [Mu] [UNS]		YES
13.9. About[Cd] [S] [St]			
14. Coverage [O] [Mu] [St]			
15. Rights [Cd] [Mu] [US]			
16. History* [M] [Mu] [N]	16.1. Event Date [M] [S] [D]		YES
	16.2. Author [M] [Mu] [St]		YES
	16.3. Reason [M] [S] [St]		YES
	16.4. Changes [Cd] [S] [St]		YES
17. Requirements [M] [S] [N]	17.1. Functional Requirements [M] [Mu] [St]		
	17.2. Non-functional Requirements [O] [Mu] [St]		
	17.3. Dependencies and Contributions [M] [S] [St]	17.3.1. Dependency Graph [M] [S] [U]	YES
		17.3.2. Contribution Graph [Cd] [S] [U]	YES
	17.4. Conflict identification & Guidance to Resolution [Cd] [Mu] [St]		
	17.5. Priorities Diagram [M] [S] [U]		
	17.6. Participants [M] [Mu] [St]		
18. Modelling [M] [S] [N]	18.1. Behaviour [M] [S] [N]	18.1.1. Use Case Diagram [M] [S] [U]	YES
		18.1.2. Collaboration/ Sequence Diagrams [M] [Mu] [U]	
		18.1.3. Activity/State Diagrams [O] [Mu] [U]	YES
	18.2. Structure [M] [S] [N]	18.2.1. Class Diagram [M] [S] [U]	
		18.2.2. Class Descriptions [M] [S] [U]	
		18.2.3. Relationship Descriptions [M] [Mu] [St]	YES
	18.3. Solution Variants** [O] [Mu] [U]		

DC2AP Element and their Rules of application			New
19. Resulting Context** [O] [Mu] [St]			
20. Design Guidelines** [O] [Mu] [St]			
21. Consequences [M] [S] [N]	21.1. Positive [M] [Mu] [St]		YES
	21.2. Negative [M] [Mu] [St]		YES
Rules' Acronyms			
Obligatoriness	Occurrence	Value Type	
[M] Mandatory [O] Optional [Cd] Conditional	[S] Single [Mu] Multiple	[St] String [D] Date [U] URI [N] Null [UNS] URI, number or string [US] URI and string	

* Version Control element.

** Experiences Collaboration element.

IV. EXAMPLE

In order to demonstrate the application of metadata profile proposed in this work, Table 3 presents an example that uses DC2AP to specify the well-known Fowler's analysis pattern called Organization Hierarchies, proposed in [3]. Not all DC2AP elements are presented in this specification, since some necessary information for these elements are absent in the original specification made by Fowler.

TABLE III. ORGANIZATION HIERARCHIES PATTERN SPECIFICATION

Example of use DC2AP		
1. Identifier: OrganizationHierarchies-v1		
2. Title: Organization Hierarchies		2.1. Alternative Title: Hierarquias de organização
3. Creator: Martin Fowler		
4. Subject: Companies, Hierarchy, Organizational Structure, Subsidiaries		
5. Description	5.1. Problem: There are many systems where we need to manage the hierarchy of an organization, registering its subsidiaries and linking them in accordance with the rules of hierarchy. How can we represent this process in a general and abstract way?	
	5.2. Motivation: - An organizational hierarchy has subdivisions like Operating Units, Regions, Divisions and Sales Offices. - Operating Units are divided into Regions. - Regions are divided into Divisions. - Divisions are divided into Sales Offices. - We need to provide a solution easy to be changed because organizations undergo changes in its hierarchy over the course of time.	5.2.1. Example: - A management system of a multinational company, for example, Microsoft. - A management system of a national company that has several branches scattered throughout the territory of a country.
	5.3. Context: This pattern is valuable to institutions or companies that have any subsidiaries. In some cases institutions may have more than one hierarchical organizational structure, but this is not a rule.	
6. Publisher: Lucas F. M. Vegi		
7. Contributor: Lucas F. M. Vegi		
8. Date		8.1. Created: 1997 8.2. Modified: 2012-01-15
9. Type: Analysis Pattern		9.1. Notation: UML
10. Format: JPEG and XMI		
11. Source: Party Pattern [3]		
12. Language: English		

Example of use DC2AP	
13. Relation	13.2. Is Replaced by: Organization Structure Pattern [3]
	13.4. Is Part of: Organization Structure Pattern [3]
	13.8. Complemented by: Party Pattern [3]
	13.9. About: This analysis pattern can be replaced by Structure Organization pattern, because it contains the pattern Hierarchies Organization specialized with a higher level of details, and thus to more complex organizational hierarchies the analysis pattr Organization Structure may be more suitable. The Organization Hierarchies pattern can be complemented by Party pattern because an Organization can be a specialization of Party, as well as the user of the system responsible for registering and changing the hierarchy can also be like that. Thus the Party pattern can be used as a complement of Hierarchies Organization pattern.
15. Rights:	This analysis pattern was originally published in [3].
16. History	16.1. Event Date: 1997
	16.2. Author: Martin Fowler
	16.3. Reason: Criation of this analysis pattern.
16. History	16.1. Event Date: 2012-01-15
	16.2. Author: Lucas F. M. Vegi
	16.3. Reason: Specification of this analysis pattern with the DC2AP metadata profile.
	16.4. Changes: Structuring the analysis pattern proposed by Fowler in a metadata profile. Within this process new diagrams referring to the solution presented by this pattern were proposed and the initial ideas of Fowler were reorganized into a structure that promotes its retrieval and subsequent reuse.
17. Requirements	17.1. Functional Requirements: (R1) Create Organizational Hierarchy - The user should be able to register the hierarchical levels of an organization for all its units. (R2) Alter Organizational Hierarchy - The user should be able to change the hierarchical levels of an organization whenever necessary, because as time goes on, with the expansion or contraction of it, such changes will certainly happen.
	17.2. Non-functional Requirements: (R3) Facility - User must make changes in the hierarchy of an organization in a fast and simple way . (R4) Security – Only users should be allowed to register and make changes in the hierarchy.
	17.3. Dependencies and Contributions: R2 depends on R1 because only registered hierarchies can be changed. R1 and R2 depends om R3 and R4 because it is desired that all these processes are being made only by authorized users, and in a simple way.
	17.3.1. Dependency Graph
	17.3.2. Contribution Graph
17.5. Priorities Diagram	
17.6. Participants: User	

Example of use DC2AP	
18.1. Behaviour	18.1.1. Use Case Diagram:
	18.1.2. Collaboration/Sequence Diagrams
	18.1.3. Activity/State Diagrams
18.2. Structure	18.2.1. Class Diagram
	18.2.2. Class Descriptions: - Organization: This class holds all the attributes common to all possible types of subsidiaries of an organization. - Operating Unit: This class represents the highest hierarchical level of an organization. - Region: This class represents a hierarchical level of an organization. This is the level immediately below an Operating Unit in a hierarchy. - Division: This class represents a hierarchical level of an organization. This is just below a Region in a hierarchy. - SalesOffice: This class represents a hierarchical level of an organization. This is just below a Division in a hierarchy.
18.2.3. Relationship Descriptions:	The self-relationship between the Organization class, super-class of all hierarchical levels, represents that the hierarchical levels communicate with each other. To correctly obey the hierarchical structure, there are restrictions establishing what hierarchical level each of them can communicate directly with.

Example of use DC2AP	
	18.3. Solution Variants: <pre> classDiagram class OperatingUnit class Region class Division class SalesOffice OperatingUnit "1" -- "0..*" Region Region "1" -- "0..*" Division Division "1" -- "0..*" SalesOffice </pre>
21. Consequences	21.1. Positive: - This analysis pattern represents an organizational hierarchy easy to change, thus making it useful in different contexts. - To change the hierarchical structure of an organization of this pattern, it is not necessary to change the model structure, but the subtypes and restrictions of the pattern. This makes the pattern flexible and reusable.
	21.2. Negative: - This pattern supports only a single organizational hierarchy, thereby limiting some contexts of use. - If restrictions are not well established for the context of use of this pattern, the self-relationship in it can be dangerous, allowing some hierarchical levels relate directly improperly.

V. CONCLUSIONS AND FUTURE WORK

DC2AP allows a detailed specification of the analysis patterns, since it was developed specifically for this domain. This profile was developed to be integrated into the proposal of Analysis Patterns Reuse Infrastructure (APRI) [5]. Thus it aims to solve the problem of documentation, organization, search and access to analysis patterns.

The use of DC2AP in an APRI allows the creation of digital collections of analysis patterns in the form of pattern catalogues and pattern languages. Through web services proposed by APRI, the analysis patterns specified with DC2AP can be retrieved more quickly and efficiently, offering to potential users an easier access to well-documented analysis patterns, and consequently, with greater potential for reuse.

Because it is a generic metadata standard, Dublin Core allows interoperability between data of different domains, so DC2AP, being an application profile of Dublin Core, can be combined with future works aimed at creating new Dublin Core application profiles to document other types of reusable computational artifacts.

As future works, it is intended to align current description of DC2AP to the Singapore Framework. This framework is a set of descriptive components recommended to document an application profile [23]. With this alignment, DC2AP will fit in the concept of machine-processable application profile and thus can serve as basis for the definition and implementation of web services proposed in APRI for search, visualization, application and contribution of use experience of analysis patterns.

ACKNOWLEDGMENT

This work is partially financed by Brazilian funding agencies: FAPEMIG, CNPq and CAPES. The authors also acknowledge the financial support of the company Sydle.

REFERENCES

- [1] B. Boehm, and V. Basili, "Software defect reduction top 10 list," IEEE Computer, vol. 34, n. 1, January 2001.
- [2] E. B. Fernandez, and X. Yuan, "Semantic Analysis Patterns," Proc. of the 19th Int. Conf. on Conceptual Modeling (ER 2000), LNCC vol. 1920. Springer, pp. 183-195, 2000.
- [3] M. Fowler, Analysis Patterns: reusable object models. Addison-Wesley Publishing, 1997.
- [4] N. Blaimer, A. Bortfeldt, and G. Pankratz, "Patterns in object-oriented analysis," Working Paper No. 451, Faculty of Business Administration and Economics, University of Hagen (Germany), 2010.
- [5] L. F. M. Vegi, D. A. Peixoto, L. S. Soares, J. Lisboa-Filho, and A. P. Oliveira, "An infrastructure oriented for cataloging services and reuse of Analysis Patterns," Proc. of BPM 2011 Workshops (rBPM 2011), LNBIP vol. 100, Part 4. Springer, pp. 338 – 343, 2012.
- [6] J. Nogueras-Iso, F. J. Zarazaga-Soria, and P. R. Muro-Medrano, Geographic information Metadata for Spatial Data Infrastructures: resources, interoperability and information retrieval. Springer, 2005.
- [7] DCMI - Dublin Core Metadata Initiative. [Online]. Available: <http://www.dublincore.org>
- [8] M. Pantoquilha, R. Raminhos, and J. Araújo, "Analysis Patterns specifications - filling the gaps," Proc. of the 2nd Viking PLoP. pp. 169-180, 2003.
- [9] R. Raminhos, M. Pantoquilha, J. Araújo, and A. Moreira, "A systematic Analysis Patterns specification," Proc. of the 8th International Conference on Enterprise Information Systems (ICEIS). pp. 453-456, 2006.
- [10] D. C. Hay, Data Model Patterns: convention of thoughts. Dorset House Publishing: New York, USA, 1995.
- [11] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design patterns: elements of reusable object-oriented software. Addison-Wesley Publishing, 1994.
- [12] J. Lisboa-Filho, C. Iochpe, and K. A. Borges, "Analysis Patterns for GIS data schema reuse on urban management applications," CLEI Electronic Journal, vol. 5, n. 2. pp. 01-15, 2002.
- [13] E. B. Fernandez, X. Yuan, "An Analysis Pattern for invoice processing," Proc. of the 16th Conference on Pattern Languages of Programs (PloP). pp. 01-10, 2009.
- [14] G. Meszaros, and J. Doble, "A pattern language for pattern writing," in Pattern languages of program design 3, R. C. Martin, D. Riehle, and F. Buschmann, Eds. Addison-Wesley: Boston, USA, 1997, pp. 529-574.
- [15] U.S. Library Of Congress, "MARC standards," Network Development and MARC Standards Office. 2004. [Online]. Available: <http://www.loc.gov/marc/>
- [16] ISO, Geographic information - Metadata. ISO 19115:2003, International Organization for Standardization. 2003.
- [17] NISO U.S. - National Information Standards Organization, The Dublin Core Metadata element set: an American national standard. NISO Press, 2001.
- [18] DCMI - Dublin Core Metadata Initiative, "Using Dublin Core - Dublin Core Qualifiers". 2005. [Online]. Available: <http://dublincore.org/documents/usageguide/qualifiers.shtml>
- [19] K. Coyle, and T. Baker, "Guidelines for Dublin Core Application Profiles." 2009. [Online]. Available: <http://dublincore.org/documents/profile-guidelines/>
- [20] DCMI - Dublin Core Metadata Initiative, "Dublin Core Collections Application Profile." 2004. [Online]. Available: <http://dublincore.org/groups/collections/collection-application-profile/>
- [21] U.S. Library Of Congress, "Dublin Core to MARC Crosswalk,". Network Development and MARC Standards Office. 2008. [Online]. Available: <http://www.loc.gov/marc/dccross.html>
- [22] L. F. M. Vegi, "Technical description of Dublin Core application profile to Analysis Patterns (DC2AP)". 2012. [Online]. Available: <http://purl.org/dc2ap/TechnicalDescription>
- [23] M. Nilsson, T. Baker, and P. Johnston, "The Singapore Framework for Dublin Core Application Profiles." 2008. [Online]. Available: <http://dublincore.org/documents/singapore-framework/>